INDEX

S.No	Date	Title	Sign
1	20/7/26	Study of various network commands used in Linux & Windows.	27/8/26 (8)
2	28/7/26	Study of different types of network cables	4/8/26 (9)
3	3/8/26	Cisco Packet tracer	18/8/26 (9)
4	10/8/26	Setup and configure a LAN using Switch and ethernet cables.	18/8/26 (10)
5	18/8/26	WIRESHARK	7/9/26 (9)
6	7/9/26	Error Detection and correction using Hamming Code	22/9/26 (9)
7	22/9/26	Nmap to discover live hosts using Nmap scans on the Tryhackme platform	28/9/26 (10)
8	28/9/26	End to End communication at transport layer	28/9/26 (10)
9	28/9/26	Sliding Window protocol	29/9/26 (9)
10	29/9/26	Implementation of subnetting in cisco packet tracer simulator	
11	5/10/26	Internetworking with routers in cisco packet Tracer	21/10/26
12	13/10/26	Routing at network layer	
13.	13/10/26	Implement your own ping program	
14	13/10/26	Packet sniffing using raw sockets	(0)

20/1/26

PRACTICAL-1

Study Of Various Network Commands Used In Linux And Windows.

Basic networking commands

① arp -a :- ARP is short form of address resolution protocol. It will show IP address of your computer along with IP address & MAC address of router

② hostname: This is the simplest of all TCP/IP commands. It simply displays the name of your computer

③ ipconfig/all :- This commands displays detailed configuration information about your TCP/IP connection.

④ nbstat -a :- This command helps in solving problems with NetBIOS name resolution.

⑤ netstat :- [Network Statistics] It displays a variety of statistics about computer active TCP/IP connection.

It is a command line tool for monitoring network connections both incoming and outgoing as well as viewing routing tables, interface statistics etc.

Eg:- netstat -r

⑥ nslookup :- Tool used for performing DNS lookups in Linux. It is used to display DNS details. It can operate in two modes interactive & non-interactive

Eg:- nslookup WWW.google.com.

⑦ Pathping : pathping is unique to windows and is basically a combination of the ping and tracert commands.

Pathping traces the route to the destination address then launches a 25 sec test of each router along the way, gathering statistics on the rate of data loss along each hop.

⑧ Ping : [Packet INternet Groper] - best way to test connectivity between two nodes.

ICMP [Internet Control Message Protocol] to communicate to other devices.

ping hostname (ping localhost)

ping ip address (ping 4.2.2.2)

⑨ Route : This command is used to show the IP routing table. It is primarily used to setup static routes to specific host or network via interface

Some Important Linux networking commands

① ip
ip <options> <objects> <commands>

ⓐ [root@server~]# ip address show
shows IP address assigned to an interface

ⓑ [root@server~]# ip address add 192.168.1.254/24
dev enp503
assign an IP to an interface.

ⓒ [root@server]# ip address del 192.168.1.254/24
dev enp503
To delete an IP on an address.

ⓓ [root@server~]# ip link set eth0 up
Alter the status of interface by bringing
the interface eth0

ⓔ [root@server~]# ip link set eth0 down
Alter the status of the interface by
bringing the eth0

ⓕ eth0: [root@server~]# ip link set eth0 promisc on
Alter the status of interface by enabling
promiscuous mode.

ⓖ eth0: [root@server]# ip route add default
via 192.168.1.254 dev eth0

ⓗ Add a route to 192.168.1.0/24 via gateway
at 192.168.1.254
[root@server~]# ip route add 192.168.1.0/24
via 192.168.1.254

② Ifconfig:- Tool for configuring and troubleshooting networks.

③ mtr :-

Mtr <options> hostname / IP

ⓐ [root@server ~] # Mtr google.com

Shows you the statistics, including each hop with time & loss %.

ⓑ [root @ server ~] # mtr -g google.com

show numeric IP address.

ⓒ [root @ server ~] # mtr -b google.com

show numeric IP address & hostnames

ⓓ [root @ server ~] # mtr -c 10 google.com.

set no. of pings that you want.

④ tcpdump:- designed for capturing & displaying packets.

ⓐ [root @ server ~] # dnf install -y tcpdump

ⓑ [root @ server ~] # tcpdump -D

ⓒ [root @ server ~] # tcpdump -i eth0 -c 10

capture traffic to & from the host

ⓓ [root @ server ~] # tcpdump -i eth0 src host 8.8.8.8

for traffic coming from 8.8.8.8

5. Ping :- It is a network utility used to check connectivity between two devices on a network.

ping google.com

Result:

Hence the various network commands used in Linux and windows are studied and executed.

27/12/26

28/7/26

Practical 2

Aim: To study different types of network cables.

a) Understand different types of network cables.

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Use
UTP	Category 3	10 bps	Advantages *) Cheaper in cost *) Easy to install as they have a smaller overall diameter Disadvantages *) More prone to Electromagnetic interference and noise	10 Base-T Ethernet Fast Ethernet, Gigabit Ethernet Fast Ethernet, Gigabit Ethernet
	Category 5	Up to 100 Mbps		
	Category 5e	1 Gbps		
STP	Category 6, 6a	10 Gbps	Advantages *) Shielded *) Faster than UTP *) Less susceptible to noise and interference	Gigabit Ethernet, 10G Ethernet (55m) Widely used in data centres
SSTP	Category 7	10 Gbps		

			Disadvantages *) Expensive *) Greater installation effort	Gigabit Ethernet, 10G Ethernet (100 m)
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages: *) High bandwidth *) Immune to interference *) Low Lost bandwidth *) Versatile Disadvantages *) Limited distance *) Cost *) Size is bulky	Speed of signal is used Television network, High speed internet connections
Fibre optics cable	Single mode Multi mode	~~100~~ Gbps	Advantages *) High speed *) High bandwidth *) High security *) Long distance Disadvantages *) Expensive *) Requires skilled installers	Maximum distance of fibre optic cable is around 100 meters.

Result: Hence, the different types of network cables are studied successfully

4/3/26

11/8/26 Exp: 3

Aim: To study cisco packet tracer tool installer and user interface overview

Introduction:

A simulator, as the name suggests, simulate network devices and its environment Packet tracer is an existing network design, simulation and modelling tool

*) It helps you to practice your network configuration and trouble shooting skills via computer or an android or ios based mobile services.

*) Protocols in packet tracer are coded to work and behave in the same way as they would on real hardware

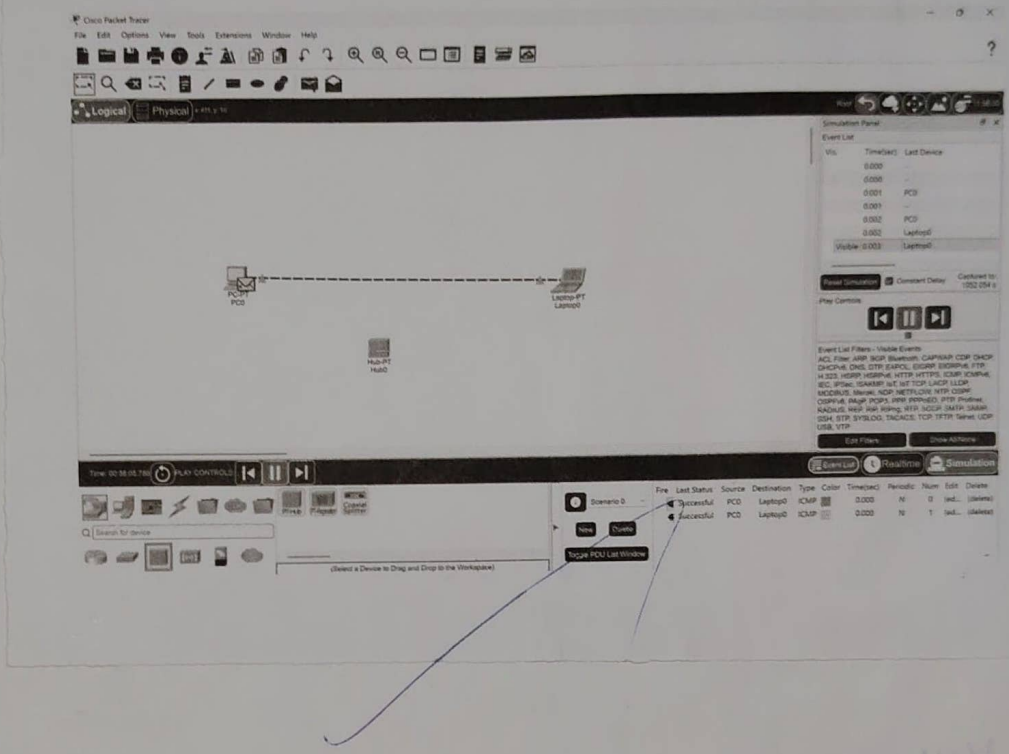
Analyze the behaviour of network devices using cisco packet tracer simulator

i) Peer to peer
ii) Switch
iii) Hub
iv) Router

① Peer to Peer:

A peer to peer network is a network in which computers communicate directly with each other without requiring a dedicated central server

Eg: Two PC's connected directly using a cable and communicating with each other

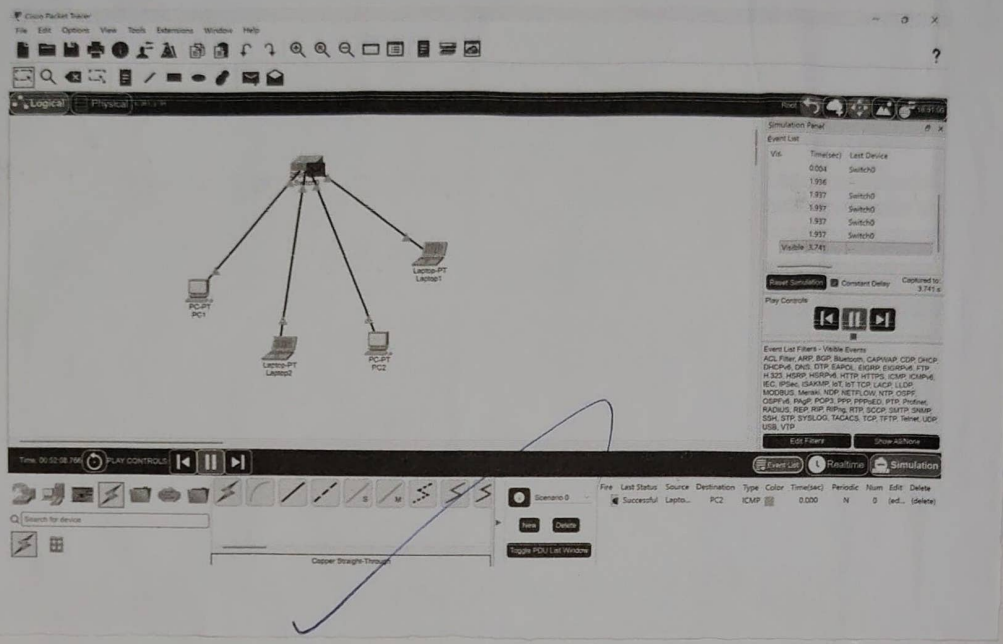


ii) Switch:-

A switch is a networking device used to connect multiple devices in a network and forward data only to the intended destination device

Eg: PC1 -> switch -> PC3

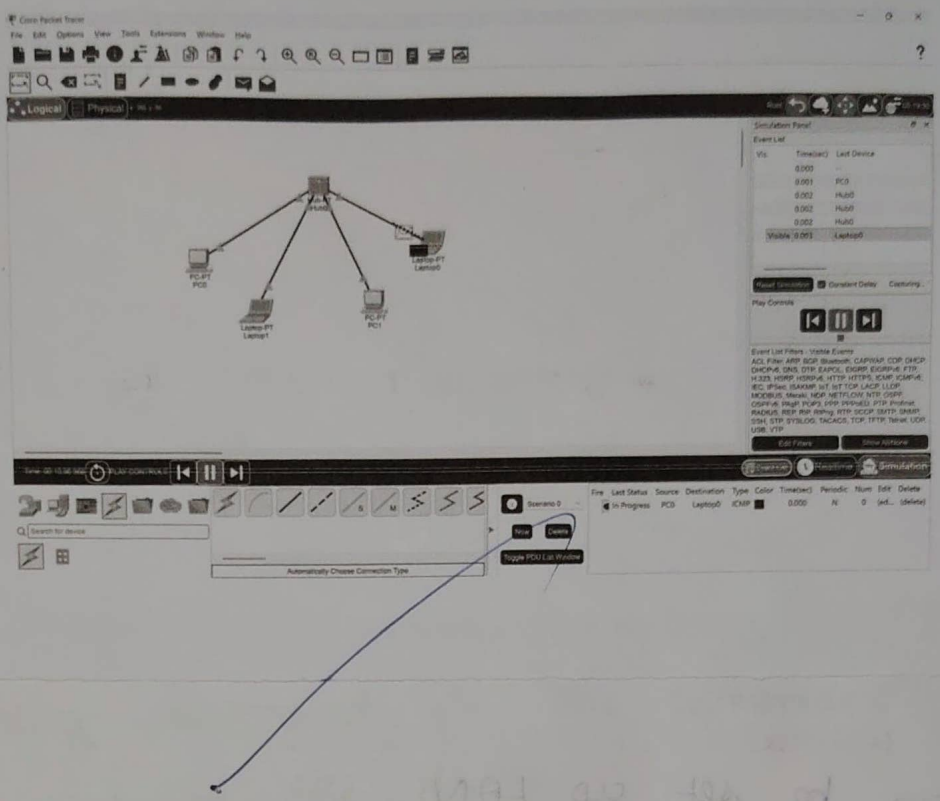
Switch knows where PC3 is and forwards the frame only towards PC3. Switch only works at the Data Link layer of the OSI model



iii) Hub :-

A hub is a basic networking device used to connect multiple computers in a network. It receive data from one device & broadcasts it to all connected devices. Hub works mainly at physical layer of the OSI Model

Eg: If PC1 send data to PC3 through a hub, the hub sends that data to PC2, PC3, PC4 etc.



Result:

Hence, the cisco packet tracer tool and its user interface has been studied

18/8/25

10/08/26 Exercise: 4

Setup and configure a LAN using switch and ethernet cables.

What is LAN?

LAN stands for Local Area Network which is a group of computer and connected devices linked together in a small physical location like a home, school, or office building

How to set up LAN

Step 1:- Connect the Hardware

*) Place your router in a central location near a power outlet.

*) Connect your modem to the router's WAN or Internet port using an Ethernet cable

*) Plug the router into power and wait for it to turn on.

Step 2:- Connect your devices

*) Wired setup:- plug one end of an Ethernet cable into your computer's network port and the other end into one of the numbered LAN ports on the router

Step 3: Configure the router

*) Type your IP address (192.168.1.1 or 192.168.0.1) into the address bar and Press enter

*) Login using the default username and password printed on the router

Step 4: Check the connectivity between switch and other Machine by using ping command in the command prompt of the device.

Step 5:

Try to process the shared folder from others computers of the network.

Result: Wireless LAN using switch and ethernet cables are executed successfully in cisco packet tracer

10/8/22

13/08/26 Exercise :-5

Wireshark

Aim: Experiments on packet capture tool: Wireshark

To filter, capture, view, packets in wireshark tool capture 100 packets from the Ethernet : IEEE 802.3 LAN interface and save it.

Procedure:

*) Select Local Area Connection in wiresharks

*) To capture option

*) Select stop capture automatically after 100 packets

*) Then click start capture.

*) Search TCP packets in search bar

*) To see flow graph click Statistics flow graph

*) Save the packets.

1. Create a Filter to display only TCP/UDP packets, inspects the packets and provide the flow graph.

*) Select LAN connection in wireshark
*) To capture option
*) Select stop capture automatically after 100 packets.
*) Then click start capture
*) Save the packets.

2 Create a filter to display only ARP packets and inspect the packets.

Procedure :-

*) To capture option
*) Select stop capture automatically after 100 packets
*) Then click start capture
*) Search ARP packets in search bar
*) Save the packets.

3. Create a filter to display only HTTP packets and inspect the packets

Procedure :-

*) Select local Area connection in wireshark

*) Go to capture option

*) Select stop capture automatically after 100 packets.

*) Then click start capture

*) Search HTTP packets in search bar.

4. Create a file to display only DNS packets and provide the flow graph

Procedure :-

*) Go to capture ~~them~~ option

*) Select stop capture automatically after 100 packets

*) Then click start capture

*) Search DNS packets in search bar

*) To see the flow graph click Statistics flow graph

5. Create a filter to display only DHCP packets and inspect the packets.

Procedure:

*) Select Local Area connection in wireshark

*) Go to capture option

*) Select stop capture automatically after 100 packets

*) Then click start capture

*) Search DHCP packets in search bar

*) Save the packets.

Result:

Wireshark packet capture tool experiment are learned and executed successfully.

18/8/21

7/09/26 Exercise: 6

Error dectection and correction using Hamming code.

Aim:

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature

Sender :-

* Input to sender file should be a text of any length program should be convert the text to binary

* Apply hamming code concept on the binary data and add redundant bits to it.

* Save this output in a file called channel.

Code :-

```
data = input("Enter 4 bits":)
d1,d2,d3,d4 = map(int,data)
P1 = d1^d2^d4
P2 = d1^d3^d4
P4 = d2^d3^d4
code = [P1,P2,d1,P4,d2,d3,d4]
```

print ("Sender Hamming code :", "".join (map(str, code)))

Receiver side:-

*) Receive the hamming code from the sender

*) Enter the received code

*) Calculate the parity check values

*) Find the error position using the parity values

*) Correct the error by changing the bit.

*) Display the corrected hamming code

Code:-

```
r = list (map (int, input ("Enter received 7bits:")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]
error = r1 * 1 + r2 * 2 + r4 * 4
if error == 0:
    print ("No error")
else:
    print ("Error at position :", error)
    r[error - 1] ^= 1
    print ("corrected code :", "".join (map(
        str, received)))
```

Input :

Enter 4 data bits : 1011

Sender Hamming code : 0110011

~~Output~~ :

Received 7 bits : 0010011

Output :

Error at position : 2

Corrected code : 0110011

Input 2 :

Enter 4 data bits : 0010

Sender Hamming code : 0101010

Enter received 7 bits : 0101010

Output :

NO error

Result :-

Error detection and error correction using hamming code has been executed successfully

7/9/24

2/9/26 Exercise: 7

NMAP to Discover Live Hosts using Nmap scans on the TryHackme platform

Aim:-

To discover live hosts using Nmaps scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform room link:

https://tryhackme.com/room/nmap01.

Procedure:

Task 1:- Introduction

*) ARP scan: This scan uses ARP requests to discover live hosts

*) ICMP scan: This scan user ICMP requests to identify live hosts

*) TCP/UDP Ping scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

Task 2:- Subnetwork

Network segment:- It is a collection of computers linked together through a common "medium".

Subnetwork:- It is usually the equivalent of one or more networks segments linked together and configured to utilize the same router in an IP network.

*) Subnets with /16

*) Subnets with /24

Task 3 :- Enumerating Targets

We need to identify the targets we want to scan before we conduct any "action".

1) List
- <MACHINE_IP> <scanme.nmap.org> <examples>
- will scan 3 IP address.

2) Range
- 10.11.12.15-20
- will scan 6 IP addresses

3) Subnet
- <MACHINE_IP>/30
- will scan 4 IP addresses

nmap -sL <TARGETS>

Task 4 :- Discovering live hosts

1) ARP from Link layer - commonly used for Man-in-Middle-Attack (ARP spoofing and ARP poisoning).

2) ICMP from Network layer - commonly used in DDOS attacks.

3) TCP from Transport layer - SYN flooding, TCP reset, TCP session hijacking attacks.

4) UDP from Transport layer - Also used in DDOS attacks

Task 5: Nmap host Discovery using ARP

1) When a privileged user tries to scan LAN (local area network), Nmap uses ARP requests

2) To discover internet hosts without port scanning - live system use nmap -sn targets.

3) nmap -PR -sn <TARGETS>

-PR specifies that you only want an ARP scan

4) nmap -PR -sn <MACHINE> /24

5) nmap -PR -sn <ATTACKBOX IP>/24

Task 6:- Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP echo requests to discover live hosts.

2) nmap -PE -sn TARGET to check host is live

3) -PP option instructs Nmap to use ICMP timestamp requests.

nmap -PP -sn TARGET

4) -PM option can enable this scan to ICMP address Mask requests.

nmap -PM -sn TARGET

Task 7: Nmap host discovery Using TCP and UDP

1) Nmap to use TCP SYN Ping use -PS followed by the port number, range list or a combination of this options

-PS21, -PS25 targets posts 21,22,23,24,25

2) nmap -PS -sn TARGET check the TCP post is open

3) TCP ACK Ping - you must be running nmap as a privileged user.

4) nmap -PA -sn TARGET to discover the online hosts on the target's subnet.

5) UDP Ping - sending a UDP packet to an open post is unlikely to result in a response.

6) If we transmit a UDP packet to a closed UDP port, we should expect to receive an ICMP port unreachable packet

nmap -PU -sn TARGET

Task 8: Using Reverse-DNS cookup

1) Nmap is to use reverse-DNS internet hosts.

2) -R option to query the DNS server even for offline hosts.

Task 9:- Summary

ARP scan, ICMP Echo scan, ICMP timestamp scan, ICMP address Mask scan, TCP SYN Ping scan, TCP ACK Ping scan, UDP ping scan

Result:-

Nmap to discover live host using tryhackme platform was successfully executed.

22/9/25

28/8/26 Exercise: 8

End to End Communication at Transport layer:

a) Aim:- Implement echo client server using TCP/UDP packets.

Procedure:-

* Create a socket using python socket programming
* Specify the IP address and post number
* Bind the server socket to the IP address and post.
* Make the server listen for client connections
* Connect the client to the server
* Send and receive messages between client and server
* Display the received message
* Close the socket connection

Code:

Tcp-server:

```
import socket
server = socket.socket( socket.AF_INET, socket.
                        *.      SOCK_STREAM)
server.bind (('127.0.0.1', 5000))
server.listen()
print ("server waiting for connection...")
conn.addr = server.accept()
print ("Client connect")
```

~~TCP client~~:

```
While True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("Client:", data)
    conn.send(data.encode())
    conn.close()
server.close()
```

TCP Client:

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
Message = input("Enter Message:")
Client.send(Message.encode())
reply = client.recv(1024).decode()
print("server:", reply)
client.close()
```

Terminal 1 - server

```
python tcp-server.py
server is waiting for connection...
client connected
client: Hello
```

Terminal 2 - Client

```
python tcp-client.py
Enter Message: Hello
server: Hello
TCP echo is working
```

Result:

The TCP/UDP packets using echo client server socket program is executed successfully.

b)

Aim: To implement chat client server using TCP packets.

Code:

```
import socket
server = socket.socket(socket.AF_INET, socket.
                       SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)
print("Server is waiting for connection...")
conn, addr = server.accept()
print("Client connected.")
while True:
    message = conn.recv(1024).decode()
    if message.lower() == 'bye':
        print("Client ended chat:")
        break
    print("Client: " message)
    reply = input("Enter server:")
    conn.send(reply.encode())
    if reply.lower() == 'bye':
        conn.close()
server.close()
```

Chat - Client side

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.bind(('127.0.0.1', 5006))
print("Connected to server")
while True:
    Message = input("Client: ")
    if Message.lower() == "bye":
        break
    reply = conn.recv(1024).decode()
    print("server: ", reply)
    if reply.lower() == "bye":
        break
client.close()
```

Terminal 1

```
python chat-server.py
waiting for client
client connected
client : Hello
server : Hi
client : How are you?
server : Fine
```

Terminal 2

```
python chat-client.py
connected to server
client : Hello
server : Hi
client : How are you?
server : Fine
```

Result :

The chat programming using sockets by TCP client & server was implemented successfully.

28/9/25

Exp No-9

Sliding Window Protocol

Aim: To write a program to implement flow control at data link layer using sliding window protocol. Simulate the flows from one node to another

Code:

```
import time
window = int(input("Enter window size:"))
start = 0
while start < len(message)
print("SENDER")
end = start + window
if end > len(message):
    end = len(message)
sender_buffer = []
for i in range(start, end):
    frame = (i, message[i])
    sender_buffer.append(frame)
    print("Frame No:", i, "DATA:", message[i])
print("sender_Buffer:")
print(sender_Buffer)
print("\n waiting for receiver...")
time.sleep(1)
print("Receives")
expected = start
error = False
for frame in sender_buffer:
```

```
frame_no = frame[0]
data = frame[1]
print("Received Frame:", frame_no, "DATA:", data)
if frame_no == expected:
    expected = expected + 1
else
    error = True
    break
# Receiver Buffer
if (error):
    receiver_buffer = ("NACK", expected)
    print("\n Receiver Buffer:")
    print (receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("ACK", ack)
    print("\n Receiver buffer:")
    print ("receiver_buffer")
    print("\n SENDER CHECKING RESPONSE")
    response = receiver_buffer[0]
    number = receiver_buffer[1]
    if response == "ACK":
        print("ACK received :", number)
        start = number + 1
    else:
        print("NACK received :", number)
        start = number
print("TRANSMISSION COMPLETED")
```

Output:

Enter window size: 3

Enter Message: Hello

SENDER

Frame No: 0 DATA: H

Frame No: 1 DATA: e

Frame No: 2 DATA: l

Sender Buffer:

[(0, 'H'), (1, 'e'), (2, 'l')]

Waiting for receiver...

RECEIVER

Received Frame: 0 DATA: H

Received Frame: 1 DATA: e

Received Frame: 2 DATA: l

Received Buffer:

('ACK', 2)

SENDER CHECKING RESPONSE

ACK RECEIVED: 2

SENDER

Frame no: 3 DATA: l

Frame no: 4 DATA: o

sender Buffer:

[(3, 'l'), (4, 'o')]

Waiting for receiver

RECEIVER

Received frame: 3 DATA:

Received frame: 4 DATA: o

Received Buffer:

('ACK', 4)

SENDER CHECKING RESPONSE

ACK RECEIVED: 4

Transmission completed

Result:

Sliding window protocols are learned and executed successfully

28/9

29/9/26 Exp No: 10

Implementation of subnetting in Cisco Packet tracer Simulator

Aim:

To implement subnetting using Cisco packet tracer and verifying communication between the subnets.

Subnetting:

It is the process of dividing large network into smaller logical networks called subnets. It helps in efficient utilization of IP address, reduces unnecessary network traffic and improve network management.

Algorithm:

*) Start cisco packet tracer
*) Place one router, two switches and four PCs in the workspace
*) Connect 2PC to the switch and both switches to the router
*) Assign static IP address and subnet masks to all PCs
*) Configure IP address on the router information connected to the 2 subnets.
*) Configure default gateway for each PC
*) Check connectivity between PCs with same subnet using Ping command
*) Test communication between PCs belonging to different subnets.

*) Disconnect lots disable the router interfaces and repeat communication test

*) Observe and record result

Procedure

Step ①: Design Network Topology

*) Connect PC0 & PC1 to switch 0

*) Connect PC2 & PC3 to switch 1

Step ②: Assign IP address.

Device	IP address	Subnet Mask	Default Gateway
PC0	192.168.1.2	255.255.255.0	192.168.1.1
PC1	192.168.1.3	255.255.255.0	192.168.1.1
PC2	192.168.2.2	255.255.255.0	192.168.2.1
PC3	192.168.2.3	255.255.255.0	192.168.2.1

Step ③: Configure the Router

*) Open CLI

*) Terminal

enable

configuration terminal

interface gigabitEthernet 0/0

ip address 192.168.1.1 255.255.255.0

no shutdown

exit

interface gigabit Ethernet 0/1
ip address 192.168.2.1 255.255.255.0
no shutdown
exit

End

Step ④ : Test communication between same subnet.

PC0, Ping 192.168.1.3

Check PC1 responds

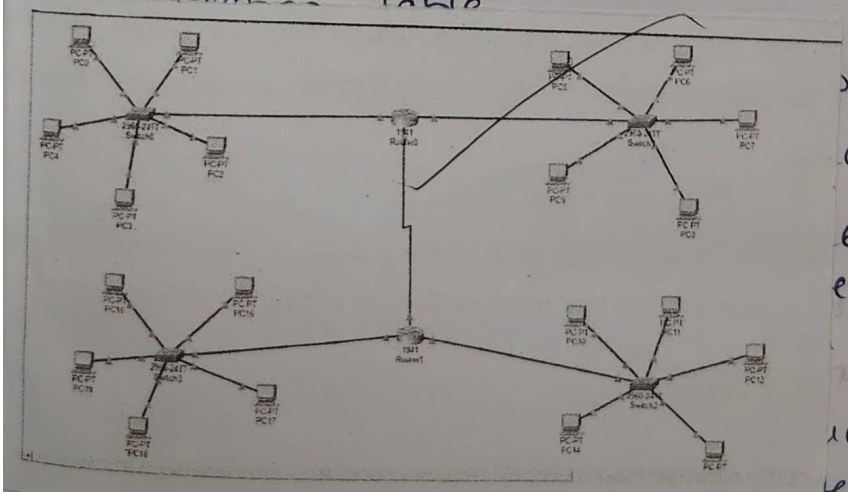
Step ⑤ : Test communication between different subnet.

PC0, ping 192.168.2.2

Check whether PC2 responds

Step ⑥ : Test communication without router

* Disconnect router links
* Attempts to ping PC2 from PC0 again.



ected Result.
ccessful
essful when
er is configured
connected.
ccessful when
er is disconnected

Result: Thus the subnetting was implemented successfully using Cisco Packet Tracer.

5/10/20 Exp No : 11

Internetworking with Router in Cisco Packet tracer simulator

11a) Internetworking using Router

Aim: To design and configure simple internetwork using router in Cisco Packet tracer

Algorithm:

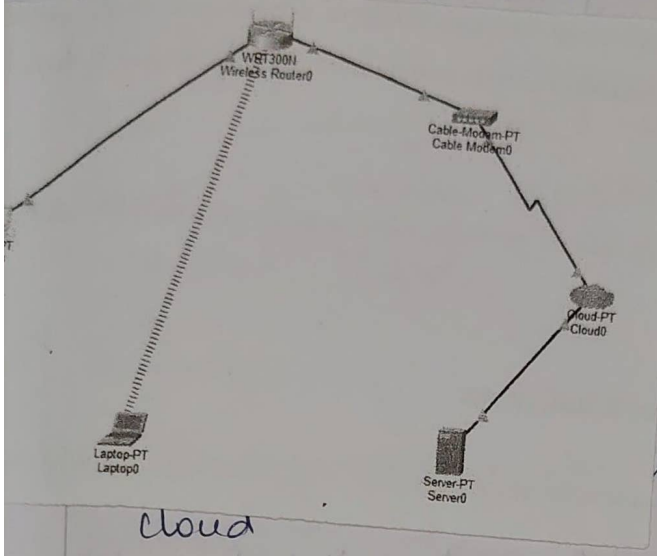
*) Place router & two PCs in packet tracer
*) Connect PCs to router using straight through cables.
*) Assign IP address and subnet masks to PC and router interface
*) Configure router using CLI and enable them
*) Use ping to test connectivity

Procedure:

*) Open packet tracer and create network topology
*) Configure router interface with IP address
*) Assign IP address to the PC in different network
*) Enable router interface using no shutdown
*) Test communication between PCs

Result: The internetwork was configured successfully & connectivity was tested

11b) Wireless Router, DHCP server and Internet



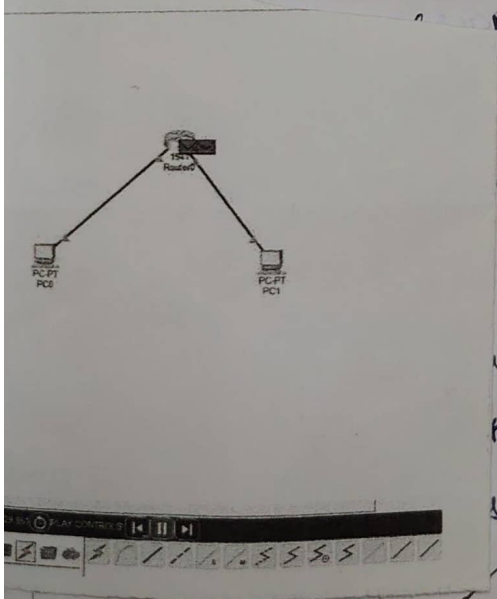
nfigure an internetwork
, DHCP server and

server, PCs and internet

Cloud

*) Configure the wireless network and DHCP service
*) Set server IP address and DNS settings.
*) Connect device and obtain IP address automatically
*) Test connectivity.

Procedure.



opology
nd configure wireless router
with IP and DNS settings.
wireless network
ddress
ty using ping
sing ping uisio.com.

Result: The wireless Internetwork was configured and, DHCP and network connectivity was tested.

21/11

11, c) Internetwork using switch, router and Ethernet cables.

Aim: To design and configure an internetwork using switches, router & ethernet cables.

Algorithm:

*) Place routers, switch and PC in cisco
*) Connect PCs to switch and switch to router
*) Assign IP address, subnet masks and default gateways
*) Configure and enable the router interface
*) Test connectivity using ping.

Procedure:

*) Create network topology
*) Connect devices using ethernet cables
*) Configure IP address
*) Enable router interface
*) Verify communication

Result: The internetwork was configured successfully and connectivity was tested

13/10/26 Exp No: 12

Routing Information Protocol

Aim: To simulate a static routing configuration using Cisco packet tracer

Procedure:

*) Open Cisco packet tracer and create the given network topology

*) Connect 3 routers and PC's using suitable cables

*) Configure IP addresses for routers and PC's

*) Enable all routers interface using no shutdown

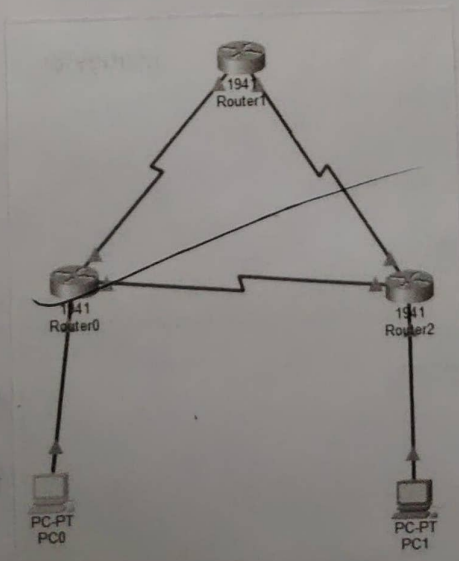
*) Configure static routers on router 0, router 1 and router 2 using IP Mobile

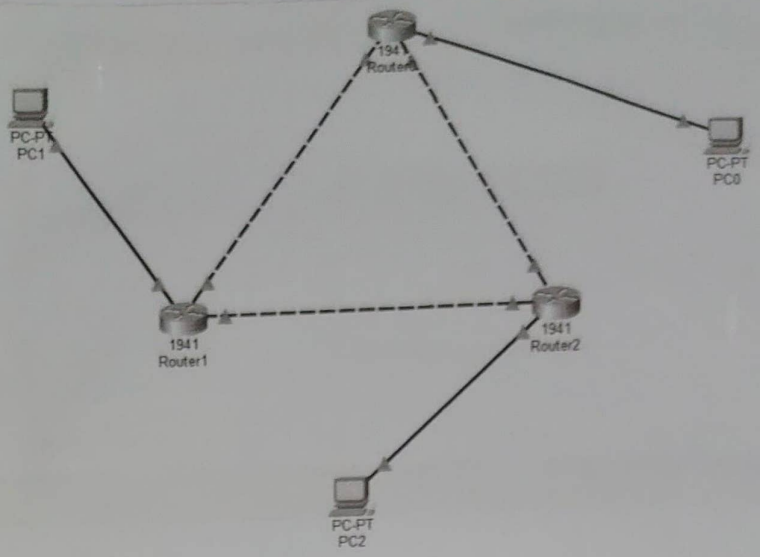
*) Set main and backup router using different administrative distance value

*) Test connectivity

*) Use tracer to verify the routing path

*) Thus a static routing is successfully configured and verified






```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2   11 ms      6 ms      5 ms      192.168.1.250
  3   29 ms      26 ms     25 ms     192.168.1.245
  4   22 ms      28 ms     20 ms     20.0.0.2

Trace complete.
```

```
Router>enable
Router#show ip route static
S     30.0.0.0/8 [20/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S     30.0.0.0/8 [10/0] via 20.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2
```

Result: Routing Information protocol successfully configured and dynamic routing between PC0 & PC1 was verified.

13/10/26 Exp no: 13.

Implement your own ping program.

Aim:

To write a python program to implement the ping utility and check network connectivity.

Algorithm:

*) Get destination IP address from user.
*) Send an ICMP echo request using the system ping command
*) Receive the response from the destination
*) Display whether host is reachable
*) Stop the program.

Code:

```
import subprocess
import platform
ip = input("Enter destination address:")
if platform.system().lower() == "windows":
    command = ["ping", "-n", "4", ip]
else:
    command = ["ping", "-c", "4", ip]
result = subprocess.run(command, capture_output=
                        True, text=True)
print(result.stdout)
```

```
If result.returncode == 0:
    print ("Host is reachable")
else
    Print ("Host is unreachable
```

O/P :-

IP :- Enter destination IP address :-
8.8.8.8

O/P :

Reply from 8.8.8.8
Host is reachable.

Result :- Thus, the python program to check network connectivity using ping utility was executed successfully

13/10/26 Exp No: 14.

Packet Sniffing using Raw Sockets.

Aim:
To write python program using raw sockets to capture network packets and display their source IP destination IP and protocol

Algorithm:-

*) Create raw sockets
*) Capture incoming IP packets.
*) Extract IP header from each packet
*) Identify source IP address & destination IP address
*) Identify protocol used
*) Display packet information

Code.

```
import socket
import struct
try:
    sniffer = socket.socket(socket.AF_INET,
                            socket.SOCK_RAW,
                            socket.IPPROTO_IP)
except OSError as e:
    Print("Socket creation failed :", e)
    print("Run with administrator/root
          priviledges".)
    raise SystemExit
print("Packet Sniffer started. Press ctrl+C to stop")
```

```
try:
    while True:
        packet, address = sniffer.recvfrom(65535)
        if len(packet) < 20:
            continue
        version_ihl = packet[0]
        version = version_ihl >> 4
        ihl = (version_ihl & 0xF) * 4
        if version != 4 or len(packet) < ihl:
            continue
        ttl = packet[8]
        protocol = packet[9]
        src_ip = socket.inet_ntoa(packet[12:16])
        dst_ip = socket.inet_ntoa(packet[16:20])

        protocols = {1: "ICMP"
                     6: "TCP"
                     17: "UDP"
                    }
        print("\n Packet captured")
        print("Source IP:", src_ip)
        print("Destination IP:", dst_ip)
        print("Protocol:" protocols.get(protocol,
                                        str(protocol)))
        print("TTL:", ttl)
        print("Packet size:", len(packet))

except KeyboardInterrupt:
        print("\n Packet Sniffing stopped.")

finally:
        sniffer.close()
```

O/P:

Packet Sniffer started.

Press ctrl +C to stop

Packet captured

Source IP : 192.168.1.10

Destination IP : 192.168.1.1

Protocol : TCP

TTL : 64

Packet size : 60

Result: Thus, the python program using raw sockets was ~~independ~~ implemented to capture network packets executed successfully

21/10/20